



Advertisement: Support JavaWorld, click here!

December 2000

HOME

FEATURED
TUTORIALS

COLUMNS

NEWS &
REVIEWS

FORUM

JW
RESOURCES

ABOUT
JW

Validation with Java and XML Schema, Part 4

Build Java representations of schema constraints and apply them to Java data

Summary

Taking validation beyond simple if-then-else structures, XML Schemas can provide a better way to validate data in Java applications. This final part in a four-part series continues to build the validation framework and fill in the blanks left from the previous article. You'll parse the XML Schema, build up Java representations of the schema's constraints, and learn how to apply those constraints to data in an application. (3,500 words)

By Brett McLaughlin

Validation. It still makes my stomach queasy to say the word; I often have nightmares of sitting at my desk, drooling on my keyboard as I type in hundreds and thousands of lines of tedious JavaScript to validate my HTML forms. My memory then moves on to more boredom, trying to put up with the lines upon lines of servlet code that handled specific constraints and often completely obscured the true purpose of my servlet code. And unless you're new to Java, you certainly share my pain.

Read the whole "Validation with Java and XML Schema" series:

- [Part 1. Learn the value of data validation and why pure Java isn't the complete solution for handling it](#)
- [Part 2. Use XML Schema for constraining Java data](#)
- [Part 3. Parsing XML Schema to validate data](#)
- [Part 4. Build Java representations of schema constraints and apply them to Java data](#)

This series of articles, and the code included, seeks to at least limit, if not completely stop, the drool and boredom, the hours spent looking out a window (or a cubicle wall!) wishing that you were outside playing hockey or listening to the Dave Matthews Band in concert. I'm thrilled at the response so far, and am glad you've come along for the ride. Before diving into this final article in the series, let me give the newcomers a little history.

The first trilogy

Well, I'm happy to see you've made it this far. As an author, there is a certain wariness about

moving beyond a series that has more than three parts. It's the "trilogy" fear, I suppose; other than *Star Wars*, there aren't many trilogies that have successful fourth outings. I mean, did any of you *really* anxiously await *Friday the 13th*, Part IV? In any case, I am glad that you are still reading.

In [Part 1](#), I talked at a conceptual level about problems common to applications with user input and raised the issue of validation. I discussed using Java properties files as well as hard-coded solutions and the various shortcomings of those approaches. Finally, I suggested that XML Schema might provide a means to represent data constraints easily and completely and previewed the framework that is the focus of this series. In [Part 2](#), I demonstrated how you could use some simple Java classes to represent data constraints defined in an XML Schema. This laid the groundwork for building a set of these `Constraint` instances and then comparing Java data (`Strings`, `ints`, `Dates`, etc.) to these data constraints. In [Part 3](#), I rounded out the framework, providing the `SchemaParser` class to parse the XML Schema and build up these constraints. And finally, I showed you the `Validator` class, which would take data and a constraint name and return whether or not the data was valid with regard to the supplied named constraint. For those of you who haven't read these parts, I strongly recommend that you read the first trilogy of articles before moving on.

So far, we have a pretty good set of classes to help us out in validating data in applications, and particularly in servlets. But there are some things missing in the framework; while validity of data is determined, there is no means to report what problems occur when data is invalid. This will be the focus of this article. Additionally, I've yet to "put it all together" and give you a solid concrete example. I'll also do that in this article and hopefully let you better see validation in action.

Improving error reporting

The framework's biggest problem as it stands so far is the lack of error reporting. If you recall from Part 3, the `Validator` class has methods to take in a piece of data, as well as a constraint name, and indicate whether that data is valid for the supplied constraint. The method used, `isValid`, returns a boolean value: `true` if the data is valid, `false` if it is not. Simple enough, right? Well, as I mentioned in Part 3, this gives you an answer, but not a complete one. For example, there is no means to report back to the user what problems occurred. Of course, the user needs this exact information to correct problems and resubmit her data. Imagine a complicated form with 20 or 25 fields; the user fills out this form (which could take a while), submits it, and makes a mistake. The user doesn't know what mistake, and the application simply replies, "Validation error. Try again." This hardly makes the user happy; in fact, the user may just close her browser or go elsewhere for her needs. This is a recipe for failure in any line of business! We clearly need to make some improvements in this area. There are two basic approaches that I suggest: the *exception* approach, where exceptions indicate validation problems, and the *message* approach, where the validation result is a Java `String`, which may have a message indicating what validation errors occurred.

The exception approach

The most obvious solution to this problem is to use Java's exception-handling method. In this scenario, the `isValid()` method would throw an exception when validation problems occurred. To paint this more concretely, look at a new class, `InvalidDataException`, that could be thrown when data is invalid:

```
package org.enhydra.validation;

/**
 *
 * The InvalidDataException represents a problem occurring
 * from data being invalid with regard to a specific data constraint.
```

```

*
*/
public class InvalidDataException extends Exception {

    /**
     *
     * This will throw an exception that simply states that
     * some sort of incorrect/invalid data was supplied.
     *
     */
    public InvalidDataException() {
        super("Invalid Data Supplied.");
    }

    /**
     *
     * This will throw an exception that simply states that
     * some sort of incorrect/invalid data was supplied, along with a
message
     * indicating the problem.
     *
     *
     * @param message String message indicating what data failed
validation.
     */
    public InvalidDataException(String message) {
        super("Invalid Data Supplied: " + message);
    }
}

```

This exception could be thrown, along with an informative error message, by the `isValid()` method. Here's the modified method to throw this exception when a problem occurs:

```

/**
 *
 * This will validate a data value (in String format) against a
 * specific constraint and throw an exception if there is a problem.
 *
 *
 * @param constraintName the identifier in the constraints to validate
this data against.
 * @param data String data to validate.
 */
public void checkValidity(String constraintName, String data)
    throws InvalidDataException {

    // Validate against the correct constraint
    Object o = constraints.get(constraintName);

    // If no constraint, then everything is valid
    if (o == null) {
        return;
    }
}

```

```

    Constraint constraint = (Constraint)o;

    // Validate data type
    if (!correctDataType(data, constraint.getDataType())) {
        throw new InvalidDataException("Incorrect data type");
    }

    // Validate against allowed values
    if (constraint.hasAllowedValues()) {
        List allowedValues = constraint.getAllowedValues();
        if (!allowedValues.contains(data)) {
            throw new InvalidDataException("Disallowed value");
        }
    }

    // Validate against range specifications
    try {
        double doubleValue = new Double(data).doubleValue();
        if (constraint.hasMinExclusive()) {
            if (doubleValue <= constraint.getMinExclusive()) {
                throw new InvalidDataException("Value is not large
enough");
            }
        }
        if (constraint.hasMinInclusive()) {
            if (doubleValue < constraint.getMinInclusive()) {
                throw new InvalidDataException("Value is not large
enough");
            }
        }
        if (constraint.hasMaxExclusive()) {
            if (doubleValue >= constraint.getMaxExclusive()) {
                throw new InvalidDataException("Value is not small
enough");
            }
        }
        if (constraint.hasMaxInclusive()) {
            if (doubleValue > constraint.getMaxInclusive()) {
                throw new InvalidDataException("Value is not small
enough");
            }
        }
    } catch (NumberFormatException e) {
        // If it couldn't be converted to a number, the data type isn't
        // numeric anyway, as it would have already failed,
        // so this can be ignored.
    }

    // If we got here, all tests were passed
    // No return value needed
}

```

Notice that I also changed the method name to `checkValidity()` since it no longer returns a boolean value (the `isXXX()` style method names are generally reserved for methods that return

a true/false result). Instead of returning false when problems occur, an `InvalidDataException` is thrown to the calling program, with an error message indicating the problem. In the sample, I've been fairly terse, but you could make this error message more descriptive, adding the value being tested, the value or values to which it must conform, and other more extensive error details.

In fact, you could extend this even further by creating additional exceptions, all extending `InvalidDataException`. For example, consider this exception hierarchy:

- `InvalidDataException`
 - `IncorrectDataTypeException`
 - `DisallowedValueException`
 - `ValueTooLowException`
 - `ValueTooHighValueException`
 - etc...

Each of these could have convenience constructors. For example:

```
public class ValueTooLowException extends InvalidDataException {
    public ValueTooLowException(String value, String minValue) {
        super("The value supplied, " + value + ", was lower than " +
            "the minimum required value, " + minValue);
    }
}
```

With this approach, you can easily report problems through the various exceptions and provide callers with detailed information about what problems occurred. In fact, this is the approach that most of you suggested to me and supported. However, the approach causes a subtle problem. Consider the client code that would use the `Validator` class:

```
Validator validator = Validator.getInstance(schemaURL);

// Validate the various pieces of data that we got earlier
try {
    validator.checkValidity("shoeSize", shoeSize);
    validator.checkValidity("width", width);
    validator.checkValidity("brand", brand);
    validator.checkValidity("numEyelets", numEyelets);
} catch (InvalidDataException e) {
    errorMessage = e.getMessage();
}

// Report back to the client the errorMessage value
```

This looks pretty good, right? Well, only the *first* validation problem will be reported. For example, assume that the user enters a shoe size of "11" (which is legal), a width of "F" (which is not legal), a brand of "V-Form" (which is legal), and for the number of eyelets, the user accidentally enters "@2" instead of "22" (which is not legal, of course). The `checkValidity()` method returns normally from the call on the shoe size, and then an exception result from the call on the width (because "F" is not legal). The program flow moves to the exception block, and the only message reported is "The width entered, F, is not in the allowed set of values: A, B, C, D, or DD. Please enter one of these values." However, the user is not informed that her entry for the number of eyelets is invalid. So she corrects her error, resubmits the form, and gets another error on the number of eyelets. This is typically frustrating for the user and causes confusion. Why couldn't both errors be reported at the same time?

To prevent this problem, the simple block of code shown above has to become this block:

```
Validator validator = Validator.getInstance(schemaURL);

StringBuffer errorBuffer = new StringBuffer();

// Validate the various pieces of data that we got earlier
try {
    validator.checkValidity("shoeSize", shoeSize);
} catch (InvalidDataException e) {
    errorBuffer.append(e.getMessage());
}
try {
    validator.checkValidity("width", width);
} catch (InvalidDataException e) {
    errorBuffer.append(e.getMessage());
}
try {
    validator.checkValidity("brand", brand);
} catch (InvalidDataException e) {
    errorBuffer.append(e.getMessage());
}
try {
    validator.checkValidity("numEyelets", numEyelets);
} catch (InvalidDataException e) {
    errorBuffer.append(e.getMessage());
}

// Convert buffer to error message
String errorMessage = errorBuffer.toString();

// Report back to the client the errorMessage value
```

Suddenly, the code's elegance in the first snippet is completely lost! We're back to the terrible spaghetti code of Part 1, which we were trying to avoid -- all because we want to get all error messages, not just the first one that occurs. So as you can see, using exceptions doesn't work quite as well as it might seem at first. For that reason, it makes sense to move on to a better approach, where you can achieve clean coding with the same result.

The message approach

So we continue to "evolve" our approach to exception and error reporting. A better solution, it

seems, would be to actually have the `checkValidity()` method directly return the error that occurred, if any. In other words, that method would return a `String` value that indicates any problems. If no problems arise, an empty string can be returned, allowing an easy check for problems that result from a validity check.

This allows your code to simply "add up" all the error messages and return them to the application user. Here's the modified `checkValidity()` method with these changes in place:

```
/**
 *
 * This will validate a data value (in String format) against a
 * specific constraint and return an error message if there is a
problem.
 *
 *
 * @param constraintName the identifier in the constraints to validate
this data against.
 * @param data String data to validate.
 * @return String - Error message, or an empty String if no
problems occurred.
 */
public String checkValidity(String constraintName, String data) {
    StringBuffer errorMessage = new StringBuffer();

    // Validate against the correct constraint
    Object o = constraints.get(constraintName);

    // If no constraint, then everything is valid
    if (o == null) {
        return "";
    }
    Constraint constraint = (Constraint)o;

    // Validate data type
    if (!correctDataType(data, constraint.getDataType())) {
        errorMessage.append("The value supplied, ")
            .append(data)
            .append(" cannot be converted to the required type,
")
            .append(constraint.getDataType())
            .append("\n");
    }

    // Validate against allowed values
    if (constraint.hasAllowedValues()) {
        List allowedValues = constraint.getAllowedValues();
        if (!allowedValues.contains(data)) {
            errorMessage.append("The value supplied, ")
                .append(data)
                .append(" is not in the allowed set of data
types
");
            for (Iterator i = allowedValues.iterator(); i.hasNext(); ) {
```

```

        errorMessage.append("'")
                        .append((String)i.next())
                        .append("'", " ");
    }
    errorMessage.setLength(errorMessage.length() - 2);
    errorMessage.append(")")
                  .append("\n");
    }
}

// Validate against range specifications
try {
    double doubleValue = new Double(data).doubleValue();
    if (constraint.hasMinExclusive()) {
        if (doubleValue <= constraint.getMinExclusive()) {
            errorMessage.append("The value supplied, ")
                        .append(data)
                        .append(" is less than or equal to the
allowed minimum value, ")
                        .append(constraint.getMinExclusive())
                        .append("\n");
        }
    }
    if (constraint.hasMinInclusive()) {
        if (doubleValue < constraint.getMinInclusive()) {
            errorMessage.append("The value supplied, ")
                        .append(data)
                        .append(" is less than the allowed minimum
value, ")
                        .append(constraint.getMinInclusive())
                        .append("\n");
        }
    }
    if (constraint.hasMaxExclusive()) {
        if (doubleValue >= constraint.getMaxExclusive()) {
            errorMessage.append("The value supplied, ")
                        .append(data)
                        .append(" is greater than or equal to the
allowed maximum value, ")
                        .append(constraint.getMaxExclusive())
                        .append("\n");
        }
    }
    if (constraint.hasMaxInclusive()) {
        if (doubleValue > constraint.getMaxInclusive()) {
            errorMessage.append("The value supplied, ")
                        .append(data)
                        .append(" is greater than the allowed
minimum
value, ")
                        .append(constraint.getMaxInclusive())
                        .append("\n");
        }
    }
}

```



```

    } catch (NumberFormatException e) {
        // If it couldn't be converted to a number, the data type isn't
        // numeric anyway, as it would have already failed,
        // so this can be ignored.
    }

    // Return any errors found
    return errorMessage.toString();
}

```

Another nice benefit to this approach is that all violations are reported on a piece of data that violates multiple constraints. For example, a piece of data that is not in the allowed set of values, and is also too low, has both problems reported (yes, I know that's a poor example; more realistically, imagine a piece of data that is too low and also is not in the required currency format, where regular expressions are supported). The user gets the complete set of information needed to make all required corrections for all data values at one time. This results in maximum user satisfaction.

The corresponding client code is not *quite* as simple as our code from Part 3, but it's a dramatic improvement over the multiple exception handling shown earlier:

```

Validator validator = Validator.getInstance(schemaURL);

StringBuffer errorBuffer = new StringBuffer();

// Validate the data
errorBuffer.append(validator.checkValidity("shoeSize", shoeSize))
            .append(validator.checkValidity("width", width))
            .append(validator.checkValidity("brand", brand))
            .append(validator.checkValidity("numEyelets", numEyelets));

String errorMessage = errorBuffer.toString();

```

And there you have it! A fairly simple change to the `Validator` class results in a much better framework, which can now accurately and completely report errors to clients, both of the validation framework and an application as a whole. Before closing shop on this framework, I want to give you a better idea of how this framework might be used in an actual application.

The framework in action

So far, I've only shown you limited code fragments. This section focuses on demonstrating an HTML form, a servlet that receives the form input, and how the user is notified of data-submission problems. I won't go into detail about how to code servlets, or set up a servlet engine. I'll leave that to you; you can also browse around *JavaWorld's* [Server-Side Java topical index](#) for more information on the subject.

Typical usage: Servlets

So let's look at a simple HTML form that lets a user enter information about the shoe she wants to view. Of course, in a complete online store, there would be graphics and colors and so forth, but I'm keeping this example simple. Rather than this form being stored as a static document, though, it is generated upon GET requests to the `BuyShoesServlet`. You'll see why in a moment, when we look at reporting errors. First, though, here's the `doGet()` method of the `BuyShoesServlet`.

```

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class BuyShoesServlet extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        // Print out HTML form
        out.println("<HTML>");
        out.println("<HEAD><TITLE>BuyShoes.com - Get All Your Shoes Here<
/TITLE></HEAD>");
        out.println("<BODY>");
        out.println("<H2 ALIGN='center'>Select a Shoe to Look At</H2>");
        out.println("<HR WIDTH='75%' />");
        out.println("<CENTER>");
        out.println("<FORM ACTION='/servlet/BuyShoesServlet'
METHOD='POST'>");
        out.println("<TABLE WIDTH='50%' BORDER='2' CELLPADDING='10'
CELLSPACING='2'>");
        out.println("        BGCOLOR='#CECECE'>");
        out.println("<TR>");
        out.println("<TD WIDTH='50%' ALIGN='right'>Shoe Size:");
        out.println("<INPUT NAME='shoeSize' SIZE='3' /></TD>");
        out.println("<TD WIDTH='50%' ALIGN='left'>Show Width:");
        out.println("<INPUT NAME='width' SIZE='2' /></TD>");
        out.println("</TR>");
        out.println("<TR>");
        out.println("<TD WIDTH='50%' ALIGN='right'>Brand:");
        out.println("<INPUT NAME='brand' SIZE='12' /></TD>");
        out.println("<TD WIDTH='50%' ALIGN='left'>Eyelets on Shoe:");
        out.println("<INPUT NAME='numEyelets' SIZE='2' /></TD>");
        out.println("</TR>");
        out.println("<TR>");
        out.println("<TD WIDTH='100%' COLSPAN='2' ALIGN='center'>");
        out.println("<INPUT NAME='submit' TYPE='submit' value='Find this
Shoe' />");
        out.println("</TD>");
        out.println("</TR>");
        out.println("</TABLE>");
        out.println("</FORM>");
        out.println("</CENTER>");
        out.println("</BODY>");
        out.println("</HTML>");

        out.close();
    }
}

```

```
}
```

This simply displays an HTML form and allows user input. And yes, I realize that no programmer in his or her right mind would let users enter all of this information without using some select boxes and so forth; but bear with me, I'm trying to keep this article an article, and not a novel. So, the user gets to enter her information. Notice that this form actually submits data to itself, in the form of an HTTP POST request. This means that in the same servlet, you need to code the `doPost()` method, which will receive the data, perform validation, and then either continue on to another servlet (if data is valid), or let the user know of an error (if invalid data was submitted). So the key to the `doPost()` method becomes the portion of code that reacts to possible validation errors.

Handling validation errors

There's actually nothing earth shattering in this code; it simply puts into practice what I've been talking about in this series. That said, you can see that once the parameter values are obtained, they are validated, and any problems are sent back to the original `doGet()` method. I'll explain that further in a moment, but first, here's the code:

```
public void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    // Grab the user input
    String shoeSize = req.getParameter("shoeSize");
    String width = req.getParameter("width");
    String brand = req.getParameter("brand");
    String numEyelets = req.getParameter("numEyelets");

    StringBuffer targetURL = new StringBuffer();
    RequestDispatcher dispatcher;

    /** Assign this in your code to your XML Schema! */
    URL schemaURL = // Assign the location in your code

    // Validate this data
    Validator validator = Validator.getInstance(schemaURL);

    StringBuffer errorBuffer = new StringBuffer();

    // Validate the data
    errorBuffer.append(validator.checkValidity("shoeSize", shoeSize))
        .append(validator.checkValidity("width", width))
        .append(validator.checkValidity("brand", brand))
        .append(validator.checkValidity("numEyelets",
numEyelets));

    String errorMessage = errorBuffer.toString();

    // If no errors, proceed, else, report errors
    if (errorMessage.equals("")) {
        dispatcher =
            getServletContext().getRequestDispatcher("PurchaseServlet");
        // Forward on to the correct URL
        dispatcher.forward(req, res);
    } else {
```

```

        req.setAttribute("shoeSize", shoeSize);
        req.setAttribute("width", width);
        req.setAttribute("brand", brand);
        req.setAttribute("numEyelets", numEyelets);
        req.setAttribute("errorMessage", errorMessage);

        doGet(req, res);
    }
}

```

The form values are determined using the `req.getParameter()` method, then they are validated. Note that in your own code, you'll need to assign the `schemaURL` variable to the correct URL for where your XML Schema is stored. Once this is set up, validation begins, and any error messages are saved. Finally, the method checks if an error has occurred. If not, it forwards the request on to the `PurchaseServlet` (not shown here), which will handle buying the selected shoe. If errors have occurred, the error message is assigned to an attribute on the `HttpServletRequest` object. The user-submitted values are assigned the same way, so the user can view the form with any error message. Finally, the program flow goes back to the `doGet()` method.

Of course, currently, the `doGet()` method doesn't use this extra information. We need to modify the method to display any data sent back to it (from a failed attempt to view a shoe), including the error message. Here's the modified method that will display these items:

```

public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {

    res.setContentType("text/html");
    PrintWriter out = res.getWriter();

    // Get any existing values
    String shoeSize = (String)req.getAttribute("shoeSize");
    if (shoeSize == null) {
        shoeSize = "";
    }
    String width = (String)req.getAttribute("width");
    if (width == null) {
        width = "";
    }
    String brand = (String)req.getAttribute("brand");
    if (brand == null) {
        brand = "";
    }
    String numEyelets = (String)req.getAttribute("numEyelets");
    if (numEyelets == null) {
        numEyelets = "";
    }

    // Print out HTML form
    out.println("<HTML>");
    out.println("<HEAD><TITLE>BuyShoes.com - Get All Your Shoes Here<
/TITLE></HEAD>");
    out.println("<BODY>");
    out.println("<H2 ALIGN='center'>Select a Shoe to Look At</H2>");

```

```

        out.println("<HR WIDTH='75%' />");
        out.println("<CENTER>");
        out.println("<FORM ACTION='/servlet/BuyShoesServlet'
METHOD='POST'>");
        out.println("<TABLE WIDTH='50%' BORDER='2' CELLPADDING='10'
CELLSPACING='2'");
        out.println("        BGCOLOR='#CECECE'>");

<b>        // Handle error message, if any
        String errorMessage = (String)req.getAttribute("errorMessage");
        if (errorMessage != null) {
            out.println("<TR>");
            out.println("<TD COLSPAN='2' ALIGN='CENTER'>");
            out.println("<B>Errors occurred in your submission:</B><BR />");
            out.println("<FONT COLOR='red'><PRE>");
            out.println(errorMessage);
            out.println("</PRE></FONT>");
            out.println("</TD>");
            out.println("</TR>");
        }</b>

        out.println("<TR>");
        out.println("<TD WIDTH='50%' ALIGN='right'>Shoe Size:");
        out.println("<INPUT NAME='shoeSize' SIZE='3' <b>VALUE='" + shoeSize
+
+ "'</b> /></TD>");
        out.println("<TD WIDTH='50%' ALIGN='left'>Shoe Width:");
        out.println("<INPUT NAME='width' SIZE='2' <b>VALUE='" + width +
+
+ "'</b> /></TD>");
        out.println("</TR>");
        out.println("<TR>");
        out.println("<TD WIDTH='50%' ALIGN='right'>Brand:");
        out.println("<INPUT NAME='brand' SIZE='12' <b>VALUE='" + brand +
+
+ "'</b> /></TD>");
        out.println("<TD WIDTH='50%' ALIGN='left'>Eyelets on Shoe:");
        out.println("<INPUT NAME='numEyelets' SIZE='2' <b>VALUE='" +
numEyelets + "'</b> /></TD>");
        out.println("</TR>");
        out.println("<TR>");
        out.println("<TD WIDTH='100%' COLSPAN='2' ALIGN='center'>");
        out.println("<INPUT NAME='submit' TYPE='submit' value='Find this
Shoe' />");
        out.println("</TD>");
        out.println("</TR>");
        out.println("</TABLE>");
        out.println("</FORM>");
        out.println("</CENTER>");
        out.println("</BODY>");
        out.println("</HTML>");

        out.close();
    }

```

So there you have it. This modified method displays any existing, presubmitted information from

the user and also displays (in red type) the error message if any exists. This, in essence, creates a looping effect until the user enters valid information. As an example, here's a screenshot from some incorrect input:



Figure 1. Click on thumbnail to view full-size image (16 KB)

And with that said, hopefully you can easily modify these examples and use the framework in your own servlets and Web applications.

What you've said

I've had a barrage of reader email concerning this series. This is of course much appreciated; it shows me that you're interested in this topic. First, I apologize to those of you whom I have not personally answered. While I try to reply to all questions and comments, I'm often faced with the choice of either answering email (of which I get all too much!), or continuing to write and work. When given those options, I almost always prefer to continue writing to reach a larger audience.

It is worth noting that I have received a significant number of emails concerning regular expressions. It seems that allowing for pattern matching and regular expression handling is a big deal to many of you. As I said, it fits easily into this framework. However, as I've also said, it would take multiple articles to cover the implementation of such a constraint. For that reason, I've chosen to focus on simpler constraints and spend more time on concepts and examples. I do hope to have some of you get involved at the open source level and add support for regular expressions, perhaps using the Apache regular expression matching code. I welcome these submissions over at the Enhydra Website (see my conclusion for more details on this). So keep those emails coming, and look for more improvements well beyond this series of articles. Thanks for all the kind words, as well!

Updates, updates, updates

As I mentioned in Part 3, often the publication of my writings is outpaced by the developments of the code that I write about. For example, JDOM continues to move forward, towards a 1.0 version, at a fast clip. As a result, code that I wrote using JDOM in, say Part 3, might need to be slightly modified to take advantage of new JDOM features or minor method name changes. Of course, this will all slow down once JDOM releases in a 1.0 final form, but until then, as they say, "them's the breaks."

In any case, I always recommend that you download *all* the code in each article and replace any older code that you might have. I also encourage you to get the latest version of JDOM, preferably from CVS; the procedure is explained in detail on the [JDOM Website](#). So get the latest code from this article (see [Resources](#)), get the latest version of JDOM, compile it all, and you shouldn't have any problems.

Conclusion

So we've made it to the end of our journey. While I'm sure we will take more together, on other subjects, I'm all talked out on validation. So I hope you enjoyed me shaking up some of your validation preconceptions, and maybe you even picked up some useful tools in the process!

The code in these articles is completely open source, under the Enhydra Public License (EPL), which essentially means you can use it however you like. Additionally, I'll be moving the code onto the Enhydra FTP server, and we'll discuss it as a group on the EnhydraEnterprise@enhydra.org mailing list, which you can sign up for at the [Enhdyra Website](#). I hope to see many of you there, making submissions and generally helping to flesh out the various ideas many of you have emailed me with regard to this framework. I'll see you all online! 

About the author

[Brett McLaughlin](#) is an Enhydra strategist at Lutris Technologies and specializes in distributed systems architecture. He is the author of *Java and XML* (O'Reilly, 2000) and is involved in technologies such as Java servlets, Enterprise JavaBeans technology, XML, and business-to-business applications. With Jason Hunter, he recently cofounded the JDOM project, which provides a simple API for manipulating XML from Java applications. McLaughlin is also an active developer on the Apache Cocoon project and the EJBoss EJB server and a cofounder of the Apache Turbine project.

Resources

- For the complete source code for this article, go to:
<http://www.javaworld.com/jw-12-2000/validation/validation.zip>
- Catch the origin of it all in Part 1 in this series:
<http://www.javaworld.com/javaworld/jw-09-2000/jw-0908-validation.html>
- Follow up by reading Part 2:
<http://www.javaworld.com/javaworld/jw-10-2000/jw-1013-validation2.html>
- Ramp up with validation by taking on Part 3:
<http://www.javaworld.com/javaworld/jw-11-2000/jw-1110-validation3.html>
- Curious about JDOM? Get into it with *JavaWorld's* recent series, "Easy Java/XML Integration with JDOM," Jason Hunter and Brett McLaughlin (May-July, 2000):
<http://www.javaworld.com/javaworld/jw-05-2000/jw-0518-jdom.html>

<http://www.javaworld.com/javaworld/jw-07-2000/jw-0728-jdom2.html>
- Get JDOM today from the Website:
<http://www.jdom.org/>
- Want to know more about XML Schema? Check out the XML Schema Activity Home Page:
<http://www.w3.org/XML/Schema.html>
- Find out more about the Enhydra application server:
<http://www.enhydra.org/>
- Check out Brett's XML Data Binding series:
<http://www-4.ibm.com/software/developer/library/data-binding1/index.html?dwzone=xml>
- Pick Brett McLaughlin's brain on all things XML by reading his book, *Java and XML*, (O'Reilly, 2000):
<http://www.oreilly.com/catalog/javaxml>
- Complete listing of all previous **Java and XML** *JavaWorld* articles:
<http://www.javaworld.com/javaworld/topicalindex/jw-ti-javaxml.html>
- Peruse ITworld.com's Java and XML forum, moderated by Mark Johnson:
<http://forums.idg.net/webx?50@@.ee6b78f>
- Sign up for the *JavaWorld This Week* free weekly email newsletter and keep up with what's new at *JavaWorld*:
<http://www.idg.net/jw-subscribe>



Advertisement: Support JavaWorld, click [here!](#)

[HOME](#) | [FEATURED TUTORIALS](#) | [COLUMNS](#) | [NEWS & REVIEWS](#) | [FORUM](#) | [JW RESOURCES](#) | [ABOUT JW](#) | [FEEDBACK](#)

Copyright © 2002 JavaWorld.com, an IDG Communications company